

VIGIL: A Dense-Convolution, Dual-Head Detector for Edge-Deployed Human-POV Threat Perception, with ARGUS Training-Run Results and VIGIL-Motion Fusion

Aras Tuğra Atay, Claude Sonnet 5

aras@mtive.tech

September 2026

Abstract

We present VIGIL, an object detector architecture built for consumer AMD GPU deployment (RDNA3) and human-worn-camera / moving-target perception, and ARGUS, the family of training-run checkpoints produced from it (v2, v3). VIGIL uses a dense-convolution-only backbone, having empirically found depthwise separable convolutions to run at approximately 1/500th the speed of dense convolutions on RDNA3 hardware despite identical FLOP counts, and a dual detection head (NMS-free one-to-one, and NMS-based one-to-many, following the split-head NMS-free design introduced for real-time detectors by YOLOv10 [5]) so that the NMS-free export target can be tracked against a conventional fallback during training. We report measured accuracy on two held-out sets (COCO val2017 [1], OpenImages val17 [2]), batch-1 latency on an RX 7900 XT, and results for VIGIL-Motion, a separate detector-agnostic movement model that fuses a per-cell motion heatmap into any detector’s box confidence. ARGUS v3 improves mean AP over ARGUS v2 by +0.031 to +0.036 on OpenImages val17 (n=3,414) at identical latency and near-identical VRAM. We also report a data-quality finding: OpenImages val17’s “person” class ground truth is measurably contaminated by non-photographic depictions (murals, statues, CGI, mannequins), a partial but real explanation for a large cross-dataset accuracy gap. We report results plainly, including where the NMS-free head still trails its NMS counterpart and where a module’s ablation shows it is more appearance-driven than motion-driven than intended.

1 Introduction

Panoptes is a human-worn-camera perception project targeting moving-target detection in unstructured environments, on consumer AMD GPU hardware rather than datacenter accelerators. This constraint — commodity RDNA3 silicon, no CUDA — ruled out directly reusing depthwise-separable-convolution architectures (MobileNet-family blocks, common in efficient YOLO variants [3, 4]) once we measured their actual throughput on the target hardware rather than trusting FLOP-count estimates.

This report documents the resulting detector, its measured performance, and what we found when we benchmarked it honestly, including results that do not flatter the system.

2 Related Work

2.1 Real-time object detectors

The YOLO family [3, 4, 5] remains the dominant architecture for real-time, single-stage detection, iterating on gradient-flow design (PGI/GELAN in YOLOv9 [4]) and on NMS-free training. YOLOv10 [5] introduced a dual-label assignment scheme — a one-to-many branch for rich supervision during training and a one-to-one branch for

NMS-free inference — which is the same o2o/o2m split VIGIL uses, adopted here for the same reason: an NMS-free head is a hard requirement for deterministic, low-latency edge inference, but training it alone, unchecked against a conventional fallback, risks silently regressing recall. DETR [6] established one-to-one bipartite matching as a way to make NMS-free detection trainable at all; VIGIL’s o2o head builds on that lineage but keeps a convolutional backbone rather than a transformer, for the throughput reasons in Section 4.

2.2 Consumer and edge hardware deployment

Most published detector benchmarks report FLOPs and CUDA latency; both are poor predictors of throughput on non-NVIDIA hardware, where kernel support for a given operation pattern (e.g. depthwise convolution) can be immature relative to dense convolution. This report’s backbone design decision (Section 3) is a direct consequence of measuring rather than assuming portability of architecture choices across vendors.

2.3 Detection benchmark quality

COCO [1] and OpenImages [2] are both widely used as held-out evaluation sets, but neither is immune to label-quality issues; the OpenImages annotation schema itself defines an IsDepiction flag for exactly the class of error

we quantify in Section 5 — drawings, statues, and other non-photographic instances of an otherwise-real class. To our knowledge this specific measurement (fraction of person-class ground truth in OpenImages val that is non-photographic) has not been previously reported for this corpus subset.

3 Architecture

3.1 VIGIL: the detector

VIGIL (class VIGIL, src/vlab/model.py) is a single convolutional detector architecture. ARGUS is not a different model — it is the naming convention for VIGIL’s successive training runs (v2, v3), each a full checkpoint of the same architecture at a later training stage. This report treats “VIGIL” as the architecture and “ARGUS v2/v3” as specific trained instances of it, and does not conflate the two.

3.1.1 Dense-convolution-only backbone

Depthwise separable convolutions, standard in mobile-oriented detectors, were measured running at approximately 1/500th the speed of dense convolutions with matched FLOP counts on the RX 7900 XT (gfx1100, ROCm). This is a hardware-specific finding, not a general claim about depthwise convolutions — it reflects a gap in RDNA3’s kernel support for that operation pattern at the time of testing. VIGIL’s backbone and neck use dense convolutions exclusively as a direct consequence.

3.1.2 Dual detection head

VIGIL trains two parallel heads on a shared backbone/neck, following the o2o/o2m split popularized by YOLOv10 [5]:

- o2o (one-to-one): NMS-free, the export/deployment target.
- o2m (one-to-many): conventional NMS-based fallback.

The o2m head’s accuracy serves as a running check on whether the NMS-free head is a viable replacement at a given training stage, rather than an untested assumption. Section 4 reports both heads on both checkpoints and both eval sets, and states plainly that o2o still trails o2m by 0.008–0.036 AP at the epoch evaluated here.

3.2 VIGIL-Motion: detector-agnostic movement fusion

VIGIL-Motion is a separate, smaller model that consumes RGB plus a short optical-flow window and outputs a per-cell movement heatmap. It has no dependency on any specific detector’s architecture — its heatmap is fused into a detector’s existing box confidences post-hoc via vlab.vigil_motion_fusion.fuse_boxes, and was verified to fuse into both VIGIL (ARGUS) and an unrelated

third-party detector (YOLOv11n [4]) using the identical heatmap output, boosting both by a comparable margin.

4 Experiments

4.1 Experimental setup

4.1.1 Evaluation sets

Two held-out sets, both distinct from the training split (coco_train2017):

- COCO val2017 [1]: 760 of 960 listed images present on disk (200 pruned earlier for disk budget) — reported n, not nominal n. Small enough that rare-class AP (truck, boat) is noisy.
- OpenImages val17 [2]: all 3,414 listed images present. The larger, more trustworthy set of the two.

4.1.2 Accuracy metric

Standard COCO-style mAP via pycocotools (COCOeval), at conf=0.001 so the full precision/recall curve is visible to the evaluator. This is a different operating point from the conf=0.30–0.45 used in live-demonstration settings; the two are not interchangeable and this report does not compare across them.

4.1.3 Implementation details

Optimized with AdamW [7], weight decay applied only to convolution/linear weights (not BatchNorm scale/bias or biases). Learning rate follows warmup, then a flat plateau, then cosine decay to a floor, repeated as warm restarts [8] at each training cycle boundary so every cycle ends at a fully annealed, independently deployable checkpoint rather than only the final one. An exponential moving average of the weights is maintained throughout and used for evaluation.

4.1.4 Latency methodology

Batch=1, 640px input, forward pass only (backbone→neck→head_o2o), on an RX 7900 XT. MIOpen auto-tunes convolution kernels on first sight of an input shape, so the first invocations measure kernel compilation rather than steady-state inference; we discard 20 warmup iterations before timing a 100-iteration mean. MIOPEN_FIND_MODE was left at its ROCm default throughout — we separately measured that setting it to FAST costs a 6.4× latency penalty on this hardware, so all reported numbers use the default. Batch-1 latency was chosen over larger-batch throughput because it is the number relevant to a real-time, single-stream deployment; batch-8 throughput is not a deployment-relevant figure for this use case and is not reported.

Table 1: COCO val2017 [1] (n=760), mean AP and person-class AP.

	ARGUS v2	ARGUS v3	Δ
mAP (o2o, NMS-free)	0.296	0.306	+0.010
mAP (o2m, NMS)	0.311	0.319	+0.008
person AP (o2o)	0.394	0.409	+0.015
person AP (o2m)	0.403	0.420	+0.017

Table 2: OpenImages val17 [2] (n=3,414) — the larger, stronger eval set.

	ARGUS v2	ARGUS v3	Δ
mAP (o2o, NMS-free)	0.253	0.289	+0.036
mAP (o2m, NMS)	0.275	0.306	+0.031
person AP (o2o)	0.180	0.200	+0.020
person AP (o2m)	0.182	0.209	+0.027

4.2 Main results

ARGUS v3 improves over v2 on both eval sets and both heads, with no single cherry-picked metric driving the result. The OpenImages gap (+0.031 to +0.036 mAP) is both larger and drawn from $4.5\times$ more images than the COCO gap, and is the number we weight more heavily.

Both checkpoints show substantially lower person AP on OpenImages (0.18–0.21) than on COCO (0.39–0.42). This is consistent across both checkpoints and both heads, so it reflects a dataset-domain difficulty gap rather than a v2-vs-v3 effect. Section 5 reports one concrete, partial cause we located for this gap.

The NMS-free (o2o) head trails the conventional (o2m) head by 0.008–0.036 AP across both checkpoints and both eval sets at this training stage (epoch 39 of a 40-epoch, two-cosine-cycle schedule). This gap did not widen from v2 to v3 on 3 of 4 comparisons, which is evidence against it being a worsening structural problem, but it is not yet closed, and we do not claim the NMS-free head is ready to fully replace the NMS fallback.

Table 3: Batch-1 latency, RX 7900 XT (gfx1100), 640px input.

Model	Params	Latency	FPS	VRAM
ARGUS v2	10.04M	3.84 ms	260.6	101 MB
ARGUS v3	10.04M	3.84 ms	260.1	130 MB
YOLOv11n[4]	2.62M	4.58 ms	218.3	69 MB

ARGUS v3’s accuracy gain over v2 (Tables 1–2) comes at identical latency and a small VRAM increase attributable to the added motion-energy head state — the accuracy improvement is effectively free.

The YOLOv11n row is included for external reference only and is not a fair speed comparison: its number is measured through `model.predict()` (letterbox, NMS, and postprocessing included), while the ARGUS num-

bers measure the raw network forward pass alone, excluding letterbox and NMS. YOLOv11n has fewer parameters (2.62M vs 10.04M) but its reported latency includes work the ARGUS number does not. We did not time ARGUS’s full inference path the same way, and explicitly do not claim “ARGUS is faster despite being larger” as a finding — that would be an apples-to-oranges artifact of measurement scope, not a real result.

4.3 Ablation: VIGIL-Motion branch contribution

Cited from a separate, prior verification pass on the shipped checkpoint (modality-dropout training), not re-run for this report:

- Held-out energy gap of +8.56 (inside-box logit mean +2.86 vs. background −5.70, test split, 8,158 samples).
- Branch ablation (zeroing one input branch on the trained checkpoint): RGB-only reproduces 86% of the joint energy gap, flow-only reproduces 63%. This is more balanced than an earlier, naively-trained checkpoint’s 81%/5% split, but the module remains more appearance-driven than motion-driven. We flag this as the most important open caveat on this module: its heatmap output is not yet demonstrated to carry independent evidence beyond what an RGB-only detector already extracts.
- Composability: the same heatmap fuses into both VIGIL/ARGUS and an unrelated third-party detector (YOLOv11n) via `fuse_boxes`, boosting both by a comparable margin on a shared real clip, supporting the detector-agnostic design intent.

5 A Data-Quality Finding: OpenImages Depiction Contamination

Motivated by the person-AP gap in Section 4, we manually inspected 36 randomly sampled OpenImages-val images containing a “person” ground-truth box. Five of 36 (approximately 14%) were not photographs of real people: a painted mural on a storefront, a CGI/3D-rendered avatar in a product advertisement, a stone fountain statue, a shop-window mannequin, and hand-drawn figures in a pencil drawing.

OpenImages’ raw annotation schema [2] includes an `IsDepiction` field specifically to flag such non-real instances. We confirmed by reading `scripts/build_o_i_val.py` that it filters `IsGroupOf==1` but never filters `IsDepiction` — an omission, not an intentional design choice. A correctly-behaving real-photo detector that declines to flag a mural as a person is scored as a false negative against this contaminated ground truth.

We report this as a real but partial explanation: approximately 14% label contamination does not account for the full $\sim 0.41 \rightarrow \sim 0.20$ person-AP drop between

COCO and OpenImages observed in Section 4. A genuine domain-difficulty gap — different crowd density, scale distribution, and scene composition between the two datasets — likely remains underneath even after this is fixed. We have not yet re-measured accuracy with IsDepiction instances excluded; doing so requires re-downloading the raw OpenImages annotation CSV (deleted from local storage for disk budget after the label .npz files were built), and is left as immediate future work.

6 Limitations

We list limitations explicitly rather than only in aggregate:

- COCO val2017 evaluation uses $n=760$ of a nominal 960 images; rare per-class AP figures on that set are noisy and not separately reported here.
- The NMS-free (o2o) head has not yet matched the NMS (o2m) head on either eval set at the checkpoint evaluated.
- VIGIL-Motion’s movement signal is demonstrably more appearance-driven than motion-driven in ablation, short of full independence from RGB appearance cues.
- OpenImages val17 person-class ground truth carries at least $\sim 14\%$ non-photographic-depiction contamination, unquantified for its effect size and unfixed as of this writing.
- The YOLOv11n reference timing in Table 3 is not scope-matched to the ARGUS timings and should not be read as a detector-vs-detector speed verdict.

7 Conclusion

ARGUS v3 is a strict improvement over ARGUS v2 on both held-out evaluation sets and both detection heads, at no additional inference latency and a small VRAM cost, making it the recommended default checkpoint for the VIGIL architecture going forward. VIGIL-Motion fuses cleanly and detector-agnostically but is not yet proven to contribute independent evidence beyond RGB appearance. We identify and report a concrete, if partial, data-quality bug affecting one of our two evaluation sets rather than treating the resulting accuracy gap as unexplained noise. Remaining open items — closing the o2o/o2m gap, re-measuring OpenImages accuracy after depiction filtering, and a scope-matched ARGUS-vs-YOLO latency comparison — are stated as such, not implied to be solved. A successor run, ARGUS v3.1, is in progress at the time of writing with a longer backbone pretraining stage and a longer, three-cycle main training schedule.

References

- [1] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, C. L. Zitnick, “Microsoft COCO: Common Objects in Context,” ECCV, 2014.
- [2] A. Kuznetsova, H. Rom, N. Alldrin, J. Uijlings, I. Krasin, J. Pont-Tuset, S. Kamali, S. Popov, M. Mallocci, A. Kolesnikov, T. Duerig, V. Ferrari, “The Open Images Dataset V4: Unified Image Classification, Object Detection, and Visual Relationship Detection at Scale,” IJCV, 2020.
- [3] C.-Y. Wang, A. Bochkovskiy, H.-Y. M. Liao, “YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors,” CVPR, 2023.
- [4] C.-Y. Wang, H.-Y. M. Liao, “YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information,” arXiv preprint arXiv:2402.13616, 2024.
- [5] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, G. Ding, “YOLOv10: Real-Time End-to-End Object Detection,” NeurIPS, 2024.
- [6] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, S. Zagoruyko, “End-to-End Object Detection with Transformers,” ECCV, 2020.
- [7] I. Loshchilov, F. Hutter, “Decoupled Weight Decay Regularization,” ICLR, 2019.
- [8] I. Loshchilov, F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” ICLR, 2017.